

Guide de test du projet - Gestion academique

Ce document explique comment lancer et verifier le projet de gestion academique en local avec Docker.

1. Prerequis

Installer sur la machine :

- Docker Desktop
- Git

Verifier que Docker fonctionne :

```
docker --version
docker compose version
```

2. Recuperer le projet

Cloner le depot :

```
git clone https://github.com/MBDS-CASA/project-final-amadiaze_tabrani.git
cd project-final-amadiaze_tabrani
```

3. Configuration des variables d'environnement

Creer le fichier .env a partir du modele :

```
copy .env.docker.example .env
```

L'application peut etre lancee sans configuration externe pour tester :

- le frontend React ;
- le backend Node.js ;
- MongoDB ;
- les comptes de test ;
- les roles ;
- les CRUD ;
- les notes ;
- les statistiques ;
- les notifications plateforme.

Pour tester les services externes, renseigner dans .env :

```
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
GOOGLE_CALLBACK_URL=http://localhost:5000/api/auth/google/callback

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=...
SMTP_PASS=...
SMTP_FROM=...

TWILIO_ACCOUNT_SID=...
TWILIO_AUTH_TOKEN=...
TWILIO_PHONE_NUMBER=...
```

4. Configuration Google SSO

Dans Google Cloud Console, ouvrir le client OAuth utilise par le projet et ajouter :

Authorized JavaScript origins :

```
http://localhost
http://localhost:5000
http://localhost:5173
```

Authorized redirect URIs :

```
http://localhost:5000/api/auth/google/callback
```

5. Lancement avec Docker

Depuis la racine du projet :

```
docker compose up -d --build
```

Verifier les conteneurs :

```
docker compose ps
```

Resultat attendu :

```
academic_mongodb    healthy
academic_backend    healthy
academic_frontend    up
```

Importer les donnees de test :

```
docker compose exec backend npm run seed
```

Tester l'API :

```
Invoke-WebRequest -Uri http://localhost/api/health -UseBasicParsing
```

Resultat attendu :

```
{"success":true,"message":"Academic Management API is running"}
```

Ouvrir l'application :

```
http://localhost
```

6. Comptes de test

Mot de passe commun :

```
password123
```

Comptes disponibles :

```
admin@academic.com
scolarite1@academic.com
teacher1@academic.com
teacher2@academic.com
student1@academic.com
student2@academic.com
student3@academic.com
```

7. Scenarios de validation

7.1 Connexion ADMIN

Compte :

```
admin@academic.com / password123
```

Verifier :

- tableau de bord global ;
- gestion des etudiants ;
- gestion des matieres ;
- gestion des UE ;

- gestion des diplomes ;
- gestion des professeurs ;
- saisie des notes ;
- creation de notifications ;
- consultation de toutes les donnees.

7.2 Connexion SCOLARITE

Compte :

scolarite1@academic.com / password123

Verifier :

- gestion administrative des etudiants ;
- gestion des matieres ;
- saisie des notes ;
- association etudiant-cours ;
- creation de notifications cibles.

7.3 Connexion STUDENT

Compte :

student1@academic.com / password123

Verifier :

- consultation des notes personnelles ;
- statistiques personnelles ;
- validation des UE ;
- notifications personnelles ;
- modification du profil et de la photo.

7.4 Connexion TEACHER

Compte :

teacher1@academic.com / password123

Verifier :

- affichage uniquement des matieres affectees ;
- affichage uniquement des UE concernees ;
- consultation des etudiants concernees ;
- saisie de notes sur ses matieres.

7.5 Google SSO

Pour tester le SSO Google :

1. Creer un etudiant avec le meme email que le compte Google teste.
2. Cliquer sur `Continuer avec Google`.
3. Se connecter avec ce compte Google.
4. Verifier que l'utilisateur accede a son espace STUDENT.
5. Verifier que les notes liees a cet email apparaissent dans la page Notes.

7.6 Notifications email et SMS

Notification email :

1. Se connecter en ADMIN ou SCOLARITE.
2. Aller dans `Notifications`.
3. Creer une notification email vers un STUDENT ou un TEACHER.
4. Verifier que l'email est reçu si SMTP est configure.

Notification SMS :

1. Verifier que le destinataire a un telephone renseigne au format international.
2. Exemple de format valide :
`+212612345678`
3. Creer une notification SMS.
4. Verifier le statut dans l'historique :

```
queued
sent
delivered
failed
```

Si aucun numero n'est renseigne, l'application conserve la notification sur la plateforme et affiche un message explicite :

SMS non envoye: aucun numero de telephone renseigne.

8. Tests backend

Depuis le dossier backend :

```
cd backend
npm install
npm run test:smoke
```

Resultat attendu :

All smoke tests passed.

9. Arret de l'application

Arreter les conteneurs :

```
docker compose down
```

Arreter et supprimer aussi la base Docker :

```
docker compose down -v
```

Si la base est supprimee, relancer ensuite :

```
docker compose up -d --build
docker compose exec backend npm run seed
```

10. Remarques

- Le fichier `.env` contient les secrets locaux et ne doit pas etre pousse sur Git.
- Le fichier `.env.docker.example` sert de modele public.
- Sans configuration Google/SMTP/Twilio, l'application reste testable, mais Google SSO, email et SMS reels ne peuvent pas fonctionner.
- L'application principale est accessible sur `http://localhost`.
- L'API backend est accessible via le proxy Docker sur `http://localhost/api`.
- Le backend est aussi expose directement sur `http://localhost:5000`.